


# 점프 투 자바

**박응용** 지음(위키독스 운영자)

이지스퍼블리싱

03

# 자바의 기초 - 자료형




**03-10** 상수 집합

**03-11** 형 변환과 final

# 03-10 상수 집합

- enum 자료형 – 서로 연관 있는 여러 개의 상수 집합을 정의할 때 사용
  - 3가지 커피를 판매하는 enum 상수 집합




**Do it!** 코딩해 보세요

```
public class Sample {
    enum CoffeeType {
        AMERICANO,
        ICE_AMERICANO,
        CAFE_LATTE
    };

    public static void main(String[] args) {
        System.out.println(CoffeeType.AMERICANO);  // AMERICANO 출력
        System.out.println(CoffeeType.ICE_AMERICANO);  // ICE_AMERICANO 출력
        System.out.println(CoffeeType.CAFE_LATTE);  // CAFE_LATTE 출력
    }
}
```

실행 결과

```
AMERICANO
ICE_AMERICANO
CAFE_LATTE
```

# 03-10 상수 집합

- enum 자료형 – 서로 연관 있는 여러 개의 상수 집합을 정의할 때 사용
  - 3가지 커피를 판매하는 enum 상수 집합
    - 반복문에서 사용

**Do it! 코딩해 보세요**

```
public class Sample {
    enum CoffeeType {
        AMERICANO,
        ICE_AMERICANO,
        CAFE_LATTE
    };

    public static void main(String[] args) {
        for(CoffeeType type: CoffeeType.values()) {
            System.out.println(type);  // 순서대로 AMERICANO, ICE_AMERICANO, CAFE_LATTE 출력
        }
    }
}
```

**실행 결과**

```
AMERICANO
ICE_AMERICANO
CAFE_LATTE
```

# 03-10 상수 집합

## enum이 필요한 이유

- 종류별로 판매된 커피의 개수를 리턴하려면 각 커피에 번호를 지정해야 함
- 각 커피의 번호를 기억하지 않아도 판매된 커피의 수를 알 수 있어 유용함

**Do it! 코딩해 보세요**

```
enum CoffeeType {
    AMERICANO,
    ICE_AMERICANO,
    CAFE_LATTE
};

/**
 * countSellCoffee는 판매된 커피의 개수를 리턴하는 메서드이다.
 * @param type 커피의 종류 (CoffeType)
 */
int countSellCoffee(CoffeType type) {
    (... 생략 ...)
}
```




- 매직 넘버를 사용할 때보다 코드가 명확함
  - 매직 넘버 - 프로그래밍에서 상수로 선언하지 않고 사용하는 숫자
- 잘못된 값을 사용해 생길 수 있는 오류를 막을 수 있음

# 03-11 형 변환과 final

## 형 변환

- 자료형을 다른 자료형으로 바꾸는 것
- 문자열을 정수로 바꾸려면 Integer 클래스 사용

**Do it! 코딩해 보세요**

```
public class Sample {
    public static void main(String[] args) {
        String num = "123";
        int n = Integer.parseInt(num);
        System.out.println(n);  // 123 출력
    }
}
```

**실행 결과**

```
123
```

# 03-11 형 변환과 final

## ▪ 형 변환

- 정수를 문자열로 바꾸려면 정수 앞에 빈 문자열("")을 더함

**Do it! 코딩해 보세요**

```
public class Sample {
    public static void main(String[] args) {
        int n = 123;
        String num = "" + n;
        System.out.println(num);  // 123 출력
    }
}
```

실행 결과

```
123
```

# 03-11 형 변환과 final

## ▪ 형 변환

- String.valueOf(정수), Integer.toString(정수)는 정수를 문자열로 바꾸어 리턴

**Do it! 코딩해 보세요**

```
public class Sample {
    public static void main(String[] args) {
        int n = 123;
        String num1 = String.valueOf(n);
        String num2 = Integer.toString(n);
        System.out.println(num1);  // 123 출력
        System.out.println(num2);  // 123 출력
    }
}
```

**실행 결과**

```
123
123
```

# 03-11 형 변환과 final

## ▪ 형 변환

- 소수점이 포함되어 있는 숫자 형태의 문자열은 Double.parseDouble이나 Float.parseFloat를 사용해 형 변환

**Do it! 코딩해 보세요**

```
public class Sample {
    public static void main(String[] args) {
        String num = "123.456";
        double d = Double.parseDouble(num);
        System.out.println(d);
    }
}
```

실행 결과

```
123.456
```

# 03-11 형 변환과 final

## ▪ 형 변환

### ▪ 정수와 실수 사이의 형 변환

**Do it! 코딩해 보세요**

```
public class Sample {
    public static void main(String[] args) {
        int n1 = 123;
        double d1 = n1;  // 정수를 실수로 바꿀 때에는 캐스팅이 필요 없다.
        System.out.println(d1);  // 123.0 출력

        double d2 = 123.456;
        int n2 = (int) d2; // 실수를 정수로 바꿀 때에는 반드시 정수형으로 캐스팅해 주어야 한다.
        System.out.println(n2);  // 소수점이 생략된 123 출력
    }
}
```

**실행 결과**

```
123.0
123
```

- 캐스팅(casting) – 실수의 소수점을 제거해 실수를 정수로 변환하는 것

# 03-11 형 변환과 final

## 형 변환

- 실수 형태의 문자열을 정수로 바꿀 때는 NumberFormatException이 발생하므로 주의

**Do it! 코딩해 보세요**

```
public class Sample {
    public static void main(String[] args) {
        String num = "123.456";
        int n = Integer.parseInt(num);  // 실수 형태의 문자열을 정수로 변환할 경우
NumberFormatException이 발생한다.
    }
}
```

  - Double.parseDouble()을 이용해 실수로 바꾼 후 사용하여 해결

# 03-11 형 변환과 final

## final

- 자료형에 값을 단 한 번만 설정할 수 있게 강제하는 키워드
- 값을 한 번 설정하면 그 값을 다시 설정할 수 없음

**Do it!** 코딩해 보세요

```
public class Sample {
    public static void main(String[] args) {
        final int n = 123;  // final로 설정하면 값을 바꿀 수 없다.
        n = 456;  // 컴파일 오류 발생
    }
}
```

# 03-11 형 변환과 final

- **final**
  - 리스트의 경우도 final로 선언하면 재할당 불가능
  - final은 프로그램을 수행하면서 그 값이 바뀌면 안 될 때 사용

**Do it!** 코딩해 보세요

```
import java.util.ArrayList;
import java.util.Arrays;

public class Sample {
    public static void main(String[] args) {
        final ArrayList<String> a = new ArrayList<>(Arrays.asList("a", "b"));
        a = new ArrayList<>(Arrays.asList("c", "d"));  // 컴파일 오류 발생
    }
}
```



# 감사합니다.